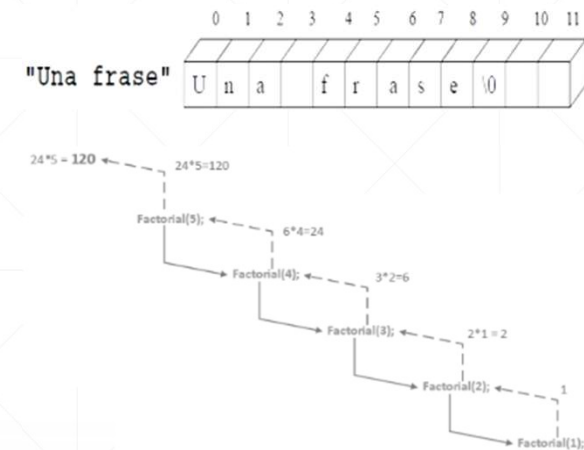




Manejo de cadenas y recursividad

Conceptos iniciales

Que caracteriza a una cadena, para ser cadena?

La barra cero /0

Como leemos una cadena?

Toda la cadena en sí o carácter por carácter.

Forma 1: Toda la cadena en sí.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

int main(){
    char cad[80]="En un lugar de la mancha";
    printf(" %30s\n",cad);
    return 0;
}
```

Forma 2: Carácter por carácter.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>

int main(){
    int i;
    char *cad="En un lugar de la Mancha";
    printf("%30s\n",cad);
    printf("Largo de la cadena:%d\n",
        strlen(cad));
    for ( i = 0; i < strlen(cad); i++){
        printf("-%c-",cad[i]);
    }
    return 0;
}
```

Ejercicio 01: Lectura de una cadena

Ingresa por teclado una cadena de caracteres, mostrarla por pantalla y determinar su longitud mediante una función definida por el usuario llamada LargoCad. Visualizar tanto la cadena ingresada como su longitud.

Ejercicio 01: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
int LargoCad(char *);

int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++){
        return i;
    }
}
```

```
int main(){
    unsigned int i;
    int l;
    char cad[81];
    printf("ingrese un Nombre:");
    scanf("%[^\n]", cad);
    printf("%30s\n", cad);
    l = LargoCad(cad);
    printf("Largo de la cadena:%d\n", l);
    for (i = 0; cad[i]; i++){
        printf("-%c-", cad[i]);
    }
    return 0;
}
```

Ejercicio 02: Memoria dinámica

Definir la cadena cad utilizando memoria dinámica, basándose en el desarrollo realizado en el Ejercicio 1.

Ejercicio 02: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
int LargoCad(char *);

int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++){
        return i;
    }
}
```

```
int main(){
    unsigned int i;
    int l;
    char *cad;
    cad = (char *)malloc(sizeof(char)*81);
    if (cad == NULL){
        printf("No hay memoria disponible");
        exit(1);
    }
    printf("ingrese un Nombre:");
    scanf("%[^\n]", cad);
    printf("%30s\n", cad);
    l = LargoCad(cad);
    printf("Largo de la cadena:%d\n", l);
    for (i = 0; cad[i]; i++){
        printf("-%-c-", cad[i]);
    }
    return 0;
}
```

Ejercicio 03: Modificación de cadenas

A partir del ejercicio anterior, desarrollar una función denominada Mayu que reciba la cadena original y una cadena destino, y devuelva en esta última la cadena original convertida completamente a mayúsculas. Realizar el llamado a la función Mayu() e implementar su definición.

Importante: Declarar siempre los prototipos de las funciones.

Ejercicio 03: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

void mayu(char[],char[]);

char mayor(char x){
    return((x>='a'&&x<='z')?x-32:x);
}

void mayu(char x[],char y[]){
    int i=0;
    while(x[i]){
        y[i]=mayor(x[i]);
        i++;
    }
    y[i]='\0';
}

int main(){
    unsigned int i;
    int l;
    char cad01[81]; char cad02[81];
    printf("ingrese un Nombre:");
    scanf("%[^\\n]", cad01);
    printf("%30s\\n", cad01);
    mayu(cad01, cad02);
    printf("%30s\\n", cad02);
    return 0;
}
```

Ejercicio 04: Impresión de cadenas

Basándose en el ejercicio anterior, implementar una función llamada Inverso que reciba la cadena original y su longitud, y muestre la cadena invertida. Realizar el llamado a la función Inverso() e implementar la función correspondiente.

Importante: Declarar siempre los prototipos de las funciones.

Ejercicio 04: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++)
    {}
    return i;
}

void inverso(char x[],int y){
    while(y>=0){
        printf("%c",x[y--]);
        //y--;
    }
}

int main(){
    unsigned int i;
    int l;
    char cad01[81]; char cad02[81];
    printf("ingrese un Nombre:");
    scanf("%[^\\n]", cad01);
    printf("%30s\\n", cad01);
    l = LargoCad(cad01);
    inverso(cad01, l);
    return 0;
}
```

Recursividad

Una definición recursiva nunca define un concepto en función de si mismo, sino siempre en función de interpretaciones más simples de dicho concepto.

Ejemplo:

Sea la definición recursiva de la suma de dos números naturales a y b

$$\text{suma}(a, b) = \begin{cases} a & \text{si } b = 0, \\ \text{sig}(\text{suma}(a, b - 1)) & \text{en otro caso} \end{cases} .$$

donde sig(b) es el número natural siguiente a b. Según esto, para calcular suma(3, 2) se realizarían los siguientes pasos:

$$\text{suma}(3, 2) = \text{sig}(\text{suma}(3, 1)) = \text{sig}(\text{sig}(\text{suma}(3, 0))) = \text{sig}(\text{sig}(3)) = \text{sig}(4) = 5.$$

Recursividad: Definición coloquial

- Se trata de funciones que se llaman a sí mismas, evitando el uso de bucles u otros iteradores.
- Una llamada recursiva sería equivalente a volver a escribir de nuevo el algoritmo recursivo invocado.

Ejercicio 05: Recursividad

Ingresa dos números por teclado y sumarlos en forma recursiva.

Ejercicio 05: Resolución Posible

/// Un programa que suma dos números en forma recursiva

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

void Suma (int a, int b, int *s){
    if (b==0)
        *s=a;
    else{
        Suma(a,b-1,s);
        *s=*s+1
    }
}
```

```
int main(){
    int i, j, k;
    //printf("ingrese un numero:");
    //scanf("%d", &i);
    //printf("ingrese otro numero:");
    //scanf("%d", &j);
    Suma (3, 2, &k);
    printf("%d\n",k);
    return 0;
}
```

Ejercicio 05: Desarrollo

```
void Suma (int a, int b, int *s)
{
    if (b==0) /* condicion de parada FALSA*/
        *s=a;
    else { /* b=2, b!=0 */
        if (b==0) /* condicion de parada FALSA*/
            *s=a;
        else
            { /* b=1, b!=0 */
                Suma(a,b-1,s); /* Se ejecuta Suma(3,0,s)
                */ *s=*s+1; /* al resultado se le sumaria 1
                */
            }
        *s=*s+1; /* al resultado se le sumaria 1 */
    }
}
```

Suma(a,b-1,s);



Ejercicio 05: Desarrollo (continuación)

```
void Suma (int a, int b, int *s)
{
    if (b==0) /* condicion de parada FALSA*/
        *s=a;
    else { /* b=2, b!=0 */
        if (b==0) /* condicion de parada FALSA*/
            *s=a;
        else { /* b=1, b!=0 */
            if (b==0) /* condicion de parada CIERTA*/
                *s=a; // s toma el valor de 3
            else { /* b=0 */
                Suma(a,b-1,s); /* NO se ejecuta */
                *s=*s+1; // NO se ejecuta */
                // FIN llamada a Suma(3,0,s), s=3
            }
            *s=*s+1; /* al resultado se le sumaria 1 */
        }
        // FIN llamada a Suma(3,1,s), s=4
        *s=*s+1 ; /* al resultado se le sumaria 1 */
        // FIN llamada a Suma(3,2,s), s=5
    }
}
```

← **Suma(a,b-1,s);**

Ejercicio 04 bis: Recursividad

Ídem el Ejercicio 04 pero utilizando recursividad, hacer una función llamada Inverso que pasándole la cadena original y el largo de la cadena pueda informar la cadena en forma invertida. Generar el llamado a la función Inverso () y realizar la función propiamente dicha.

Ejercicio 04 bis Resolución Posible

```
/// Un programa que lee una cadena de  
texto y la muestra en orden inverso  
utilizando recursividad
```

```
#include <stdio.h>  
#include <conio.h>  
#include <stdlib.h>  
#include <string.h>
```

```
void Recu(char x[],int y){  
    if(x[y])  
        Recu(x,y+1);  
    printf("%c",x[y-1]);  
}
```

```
int main(){  
    unsigned int i;  
    int l;  
    char cad[81];  
    printf("ingrese un Nombre:");  
    scanf("%[^\n]", cad);  
    printf("%30s\n", cad);  
    Recu(cad, 1);  
    return 0;  
}
```

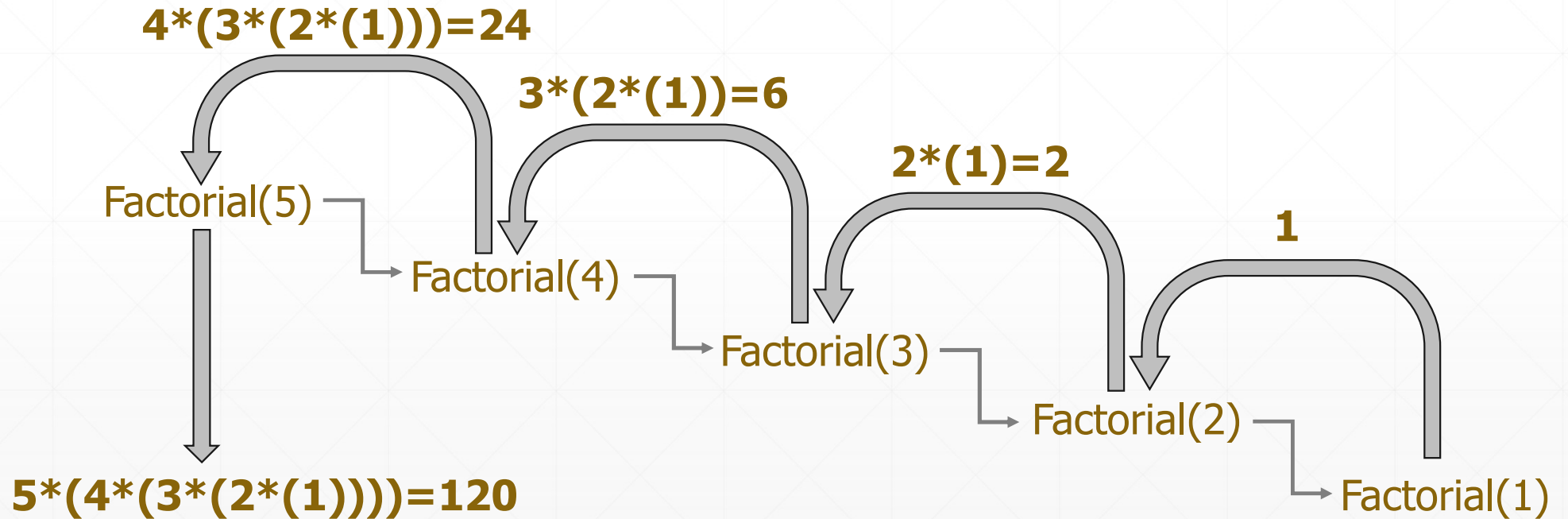
Factorial de un número: Resolución convencional

```
int main(){
    int a, b, fact = 1;
    printf ("Ingresar un numero para calcular su factorial: ");
    scanf ("%d", &a);
    for (b = a; b > 1; b--){
        fact = fact * b;
    }
    printf ("El factorial de %d es %d\n", a, fact);
    return 0;
}
```

Factorial de un número: Resolución recursiva

A continuación, veremos el comportamiento de una función recursiva a través de la resolución del factorial del número 5.

Ejemplo Factorial de 5



Factorial Resolución Posible

```
int factorial (int n) {
    if (n<=1) return 1;
    else return (n*factorial (n-1));
}

int main(){
    printf ("%d",factorial (5));
    return (0);
}
```

factorial (5)
if (5<=1) return 1;
else return (5*factorial (4))

$$4*(3*(2*(1)))=24$$

factorial (4)
if (4<=1) return 1;
else return (4*factorial (3));

factorial (3)
if (3<=1) return 1;
else return (3*factorial (2));

$$2*(1)=2$$

factorial (2)
if (2<=1) return 1;
else return (2*factorial (1));

factorial (1)
if (1<=1) return 1;

1

$$3*(2*(1))=6$$

$$5*(4*(3*(2*(1))))=120$$

Ejercicio 06: Recursividad

Ahora, sobre el mismo ejercicio de cadenas, determinemos cuántas vocales y cuántas consonantes hay. Para ello deberemos crear una función llamada `contar_vocales_consonantes` donde le tendremos que pasar dos variables de tipo entero, en las cuales deberá por parámetro, devolver la cantidad de vocales y consonantes que existe en la frase. Obviamente se tendrá que pasar la cadena para poderla trabajar.

Consultas

